

-
-
-
-
-
-
-
-
-
-

Graphs

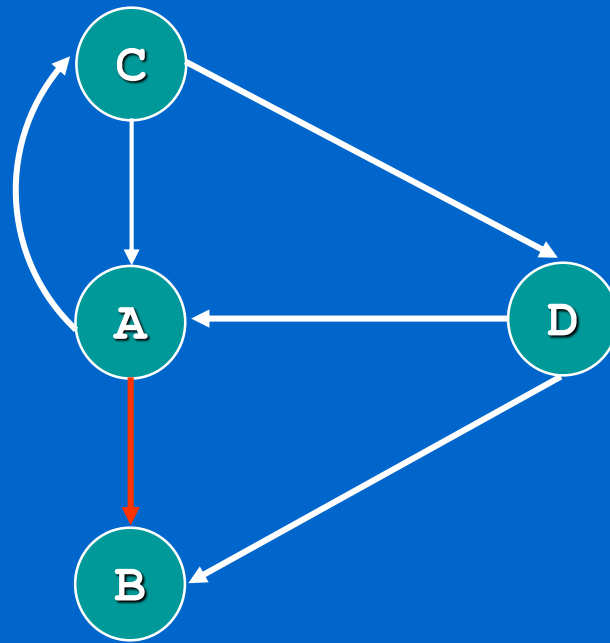
-
-
-
-
-
-
-
-
-
-

โครงสร้างกราฟ

โครงสร้างกราฟ

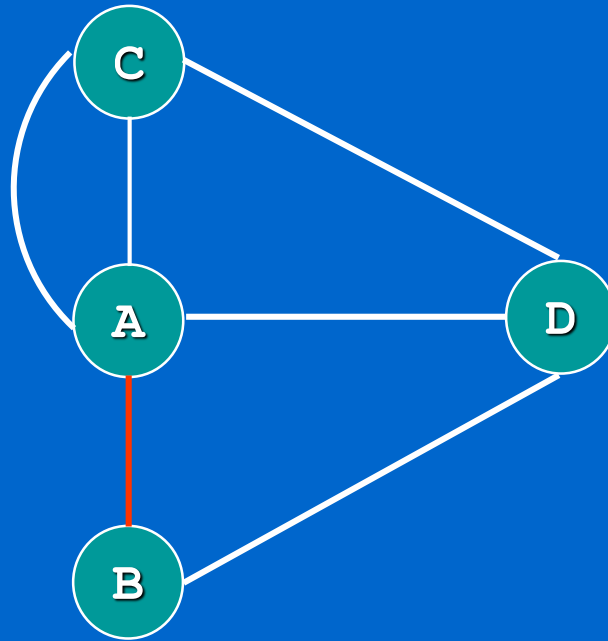
- กราฟเป็นโครงสร้างที่ประกอบด้วย**จุดยอด** (Vertex) และมี**ขอบ** (Edge) เชื่อมโยงแสดงความสัมพันธ์ระหว่าง**จุด** **ขอบ**ในโครงสร้างรูปกราฟจะมีหัวลูกศรกำกับทิศทางหรือไม่ก็ได้กราฟที่มีหัวลูกศรกำกับทิศทางเรียกว่ากราฟแบบมีทิศทาง (Directed Graph) หรือนิยมเรียกว่าไดกราฟ (DiGraph) ส่วนกราฟที่ไม่มีหัวลูกศรกำกับทิศทางก็จะเรียกว่ากราฟแบบไม่มีทิศทาง (UnDirected Graph)

กราฟแบบมีทิศทาง (Directed Graph)



- แสดงถึงไดกราฟที่มี จุดยอด A และ B แสดงความสัมพันธ์จาก A ไป B ดังนั้น AB จึงเป็นขอบของกราฟ แต่ BA ไม่เป็นขอบของกราฟ

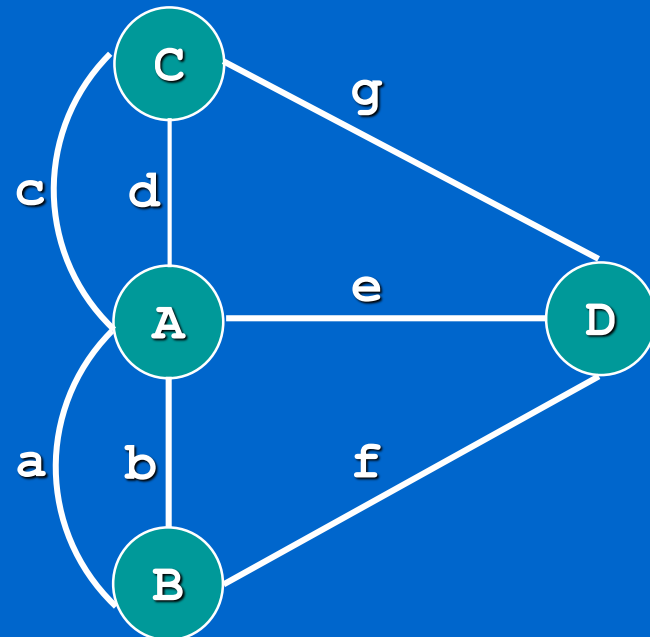
กราฟแบบไม่มีทิศทาง (UnDirected Graph)



- แสดงกราฟแบบไม่มีทิศทาง จุดยอด A และ B ไม่บ่งบอกทิศทาง เราจึงถือว่าทั้ง AB และ BA ต่างก็เป็นขอบ

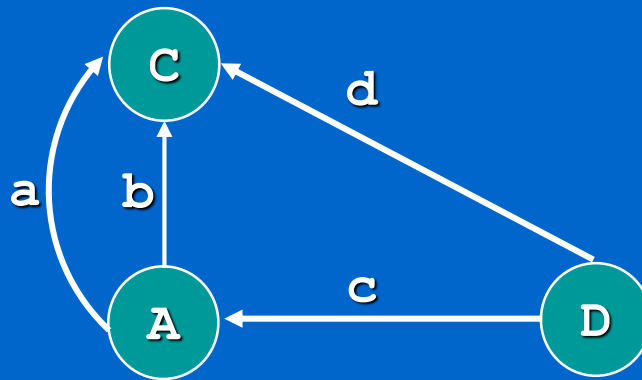
โครงสร้างกราฟ

- แสดงถึงกราฟลักษณะต่างๆ จุดยอดสองจุดใดๆ ที่มีขอบเชื่อมกันอยู่จะเรียกว่าเป็น**จุดประชิด** (Adjacent) และเส้นทางจากจุดยอดจุดหนึ่งไปยังจุดยอดอีกจุดหนึ่งเรียกว่า**ทางเดิน** (Path) อย่างเช่นทางเดินจาก A ไป C ในกราฟคือ A, C และ A, D, C



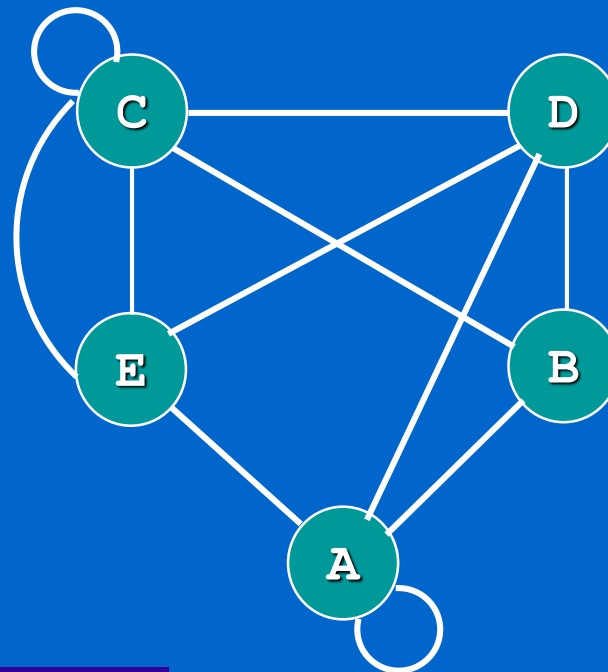
โครงสร้างกราฟ

- จำนวนขอบที่จุดใดๆ เรียกว่า **ดีกรี** (Degree) ของจุดนั้น สำหรับไดกราฟ
จำนวนขอบที่มีทิศทางพุ่งเข้าสู่จุดจะเรียกว่า **ดีกรีขาเข้า** (InDegree) และ
จำนวนขอบที่มีทิศทางพุ่งออกจากจุดจะเรียกว่า **ดีกรีขาออก**
(OutDegree) ตัวอย่างเช่น A มีดีกรีขาเข้าเท่ากับ 1 และดีกรีขาออก
เท่ากับ 2 C มีดีกรีขาเข้าเท่ากับ 3 และดีกรีขาออกเท่ากับ 0



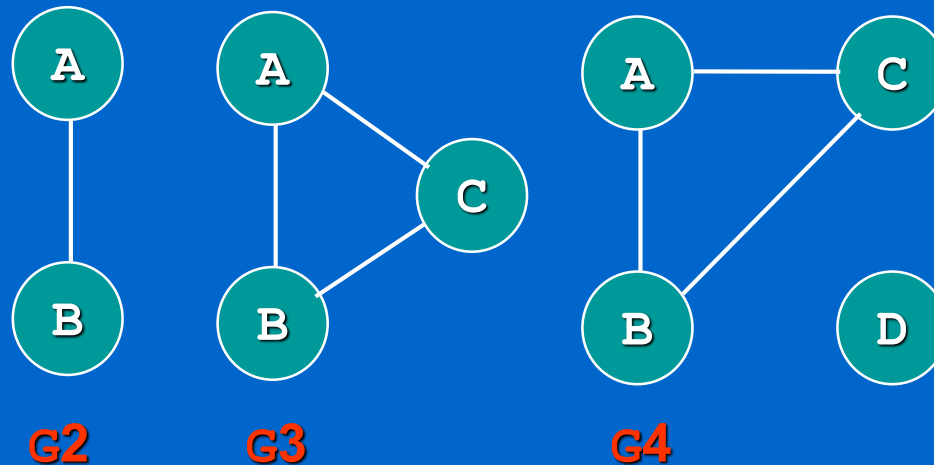
โครงสร้างกราฟ

- ถ้าจุด V ใด ๆ มีวัฏจักรในรูป V, V เราจะเรียกวัฏจักรนั้นว่า **วงรอบตัวเอง** (Self Loop) ดังเช่นที่จุดยอด A และ C ของ



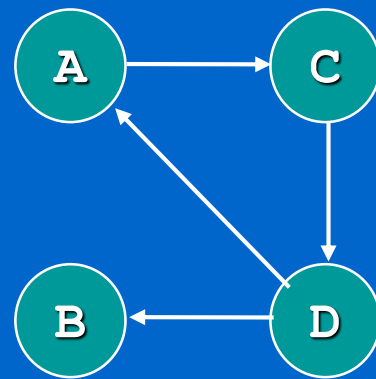
ประเภทของกราฟ

- กราฟใดๆจะมีคุณสมบัติต่อเนื่อง (Connected Graph) เมื่อมีทางเดินจากจุดยอดจุดใดๆไปยังจุดยอดจุดอื่นได้เสมอ ตัวอย่างเช่น กราฟ $G2$ และ $G3$ เป็นกราฟต่อเนื่อง ส่วน $G4$ เป็นกราฟไม่ต่อเนื่อง

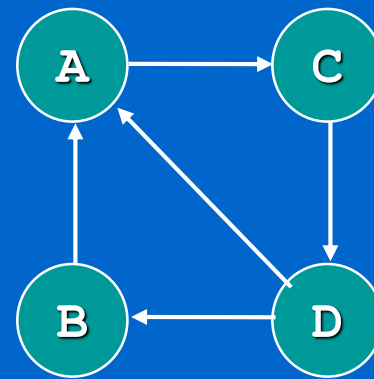


ประเภทของกราฟ

- กราฟใดๆจะเป็นกราฟต่อเนื่องแท้จริง (Strictly Connected Graph) เมื่อมีทางเดินอย่างน้อยหนึ่งเส้นทางจากจุดหนึ่งไปยังจุดอื่นใดๆ ตัวอย่างเช่น $G1$ เป็นกราฟต่อเนื่องแต่ไม่เป็นกราฟต่อเนื่องแท้จริง เพราะไม่มีทางเดินจากจุด B ไปยัง C หรือ A ในขณะที่ $G2$ เป็นกราฟต่อเนื่องแท้จริง



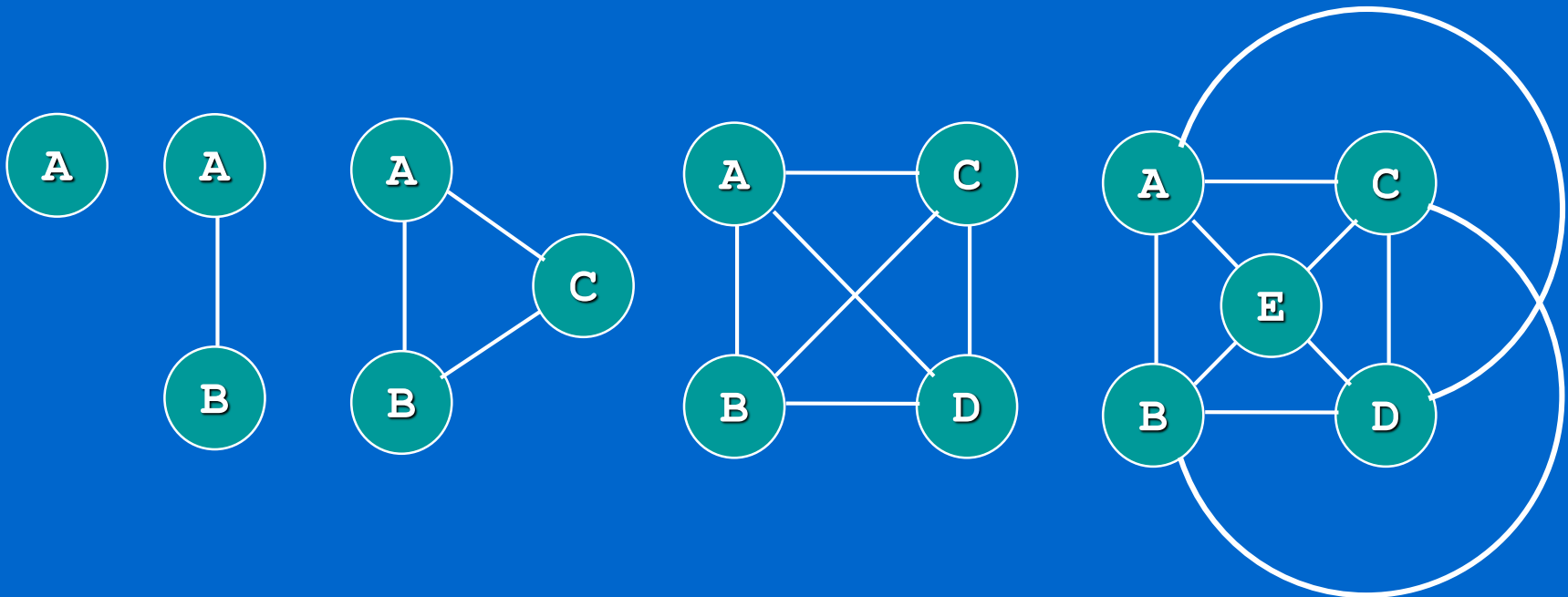
$G1$



$G2$

ประเภทของกราฟ

- กราฟที่ทุกๆจุดมีขอบไปยังจุดอื่นทุกจุดจะเรียกว่า**กราฟบริบูรณ์** (Complete Graph) กราฟบริบูรณ์ย่อมเป็นกราฟต่อเนื่องแท้จริงด้วยและจะมีจำนวนขอบเท่ากับ $n(n-1)/2$ เมื่อ n คือจำนวนจุดในกราฟ



-
-
-
-
-
-
-
-
-
-

การแทนกราฟ

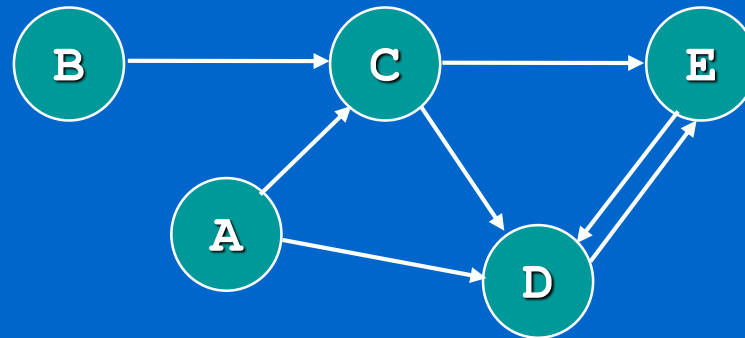
การแทนกราฟ

- การเขียนโปรแกรมเพื่อแก้ปัญหาที่เกี่ยวข้องกับกราฟ จำเป็นต้องหาวิธีการแทนกราฟด้วยโครงสร้างข้อมูลที่เหมาะสม ในที่นี้เราจะพิจารณาถึงการแทนกราฟซึ่งใช้กันอยู่โดยทั่วไป 2 แบบคือ
 - การแทนด้วยเมตริกซ์
 - การแทนด้วยลิสต์ประชิด

การแทนด้วยเมตริกซ์

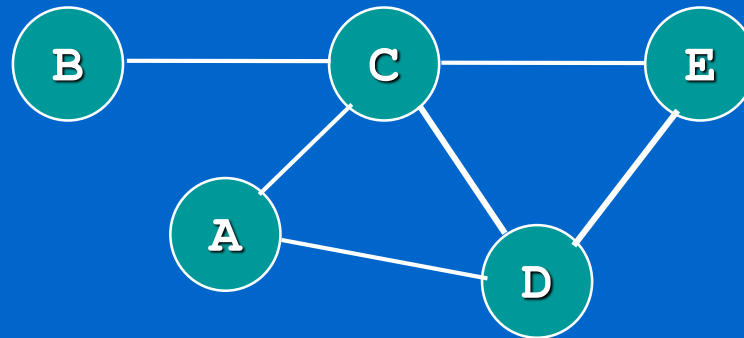
- การแทนกราฟด้วยวิธีนี้จะใช้เมตริกซ์จัตุรัสขนาด $V \times V$ หน่วย เมื่อ V คือจำนวนจุดยอดภายในกราฟ เราจะเรียกเมตริกซ์จัตุรัสที่แทนกราฟนี้ว่าเมตริกซ์ประชิด (Adjacency Matrix) เมตริกซ์ประชิดจะอยู่ในรูปอะเรย์ของ integer ขนาด $V \times V$ หน่วย หากให้ A เป็นอะเรย์ดังกล่าว องค์ประกอบ $A[x][y]$ จะมีค่าตามเงื่อนไข
1 หากมีขอบจาก x ไปยัง y
0 หากไม่มีขอบจาก x ไปยัง y

การแทนด้วยเมตริกซ์



	A	B	C	D	E
A	0	0	1	1	0
B	0	0	1	0	0
C	0	0	0	1	1
D	0	0	0	0	1
E	0	0	0	1	0

การแทนด้วยเมตริกซ์

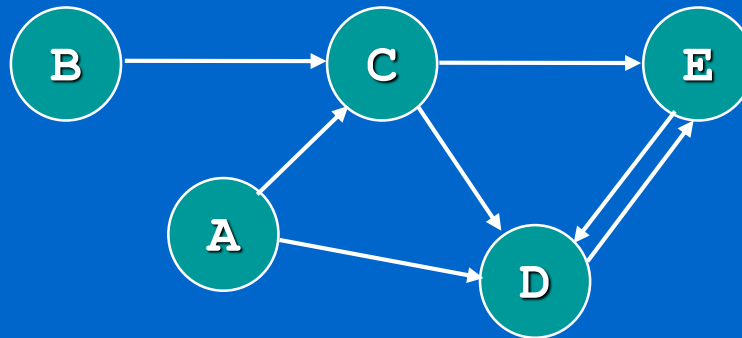


	A	B	C	D	E
A	0	0	1	1	0
B	0	0	1	0	0
C	1	1	0	1	1
D	1	0	1	0	1
E	0	0	1	1	0

การแทนด้วยเมตริกซ์

- แสดงถึงการใช้เมตริกซ์ประชิดแทนกราฟ ชื่อจุดยอดที่มีในกราฟจะถูกแปลงเป็นเลขจำนวนเต็มที่มีค่าจาก 0 ถึง $v-1$ เพื่อนำไปใช้เป็นดัชนีของอะเรย์ การแปลงชื่อจุดยอดให้เป็นเลขจำนวนเต็มอาจจะใช้ตารางเก็บชื่อจุดยอดแล้วใช้การค้นแบบทวิภาค หรือใช้ฟังก์ชันแบบแฮช วิธีดังกล่าวนี้จะช่วยให้จัดการกับกราฟโดยใช้ชื่อจริงของจุดยอดได้โดยง่าย ในกรณีที่ชื่อจุดยอดเป็นเลขจำนวนเต็มอยู่แล้ว ก็สามารถชื่อจุดยอดมาเป็นดัชนีได้ทันที

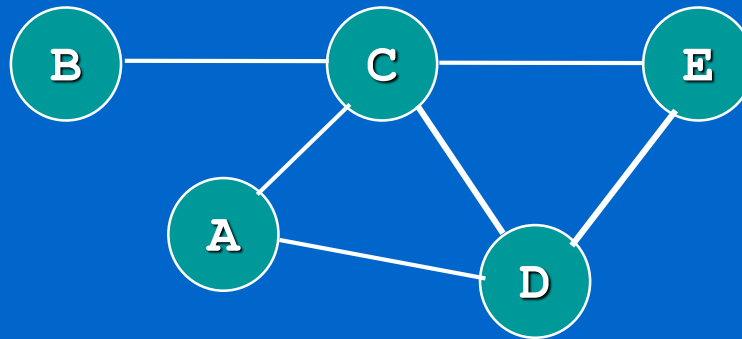
การแทนด้วยเมตริกซ์



A	0
B	1
C	2
D	3
E	4

	0	1	2	3	4
0	0	0	1	1	0
1	0	0	1	0	0
2	0	0	0	1	1
3	0	0	0	0	1
4	0	0	0	1	0

การแทนด้วยเมตริกซ์



A	0
B	1
C	2
D	3
E	4

	0	1	2	3	4
0	0	0	1	1	0
1	0	0	1	0	0
2	1	1	0	1	1
3	1	0	1	0	1
4	0	0	1	1	0

การแทนด้วยเมตริกซ์

- การแทนกราฟด้วยเมตริกซ์ประชิดช่วยให้การเข้าถึงข้อมูลในกราฟเป็นไปอย่างรวดเร็ว การตรวจหาว่า จุด x และ y ประชิดกันหรือไม่ สามารถทำได้ในเวลาอันสั้น เพราะเพียงแค่ใช้ดัชนีคำนวณหาตำแหน่ง (ในกรณีที่แทนชื่อจุดยอดด้วยเลขจำนวนเต็มและใช้ชื่อจุดยอดเป็นดัชนี) การแทนกราฟด้วยเมตริกซ์ประชิดจะเหมาะสมกับกราฟที่ไม่มีลักษณะหรือมั่ว (Sparse) เนื่องจากเมตริกซ์ต้องการเนื้อที่เก็บ V^2 หน่วย และขั้นตอนวิธีใดๆก็ตามในการประมวลผลกราฟจะมีความซับซ้อน

-
-
-
-
-
-
-
-
-
-

การแทนด้วยลิสต์ประชิด

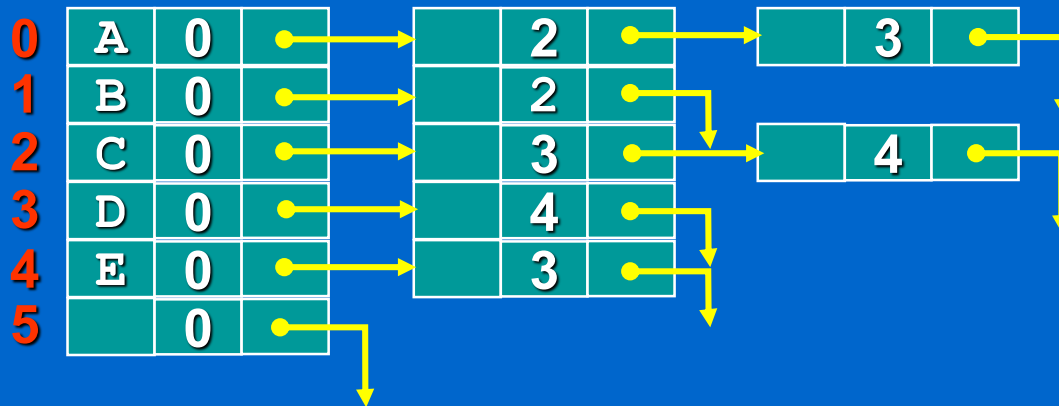
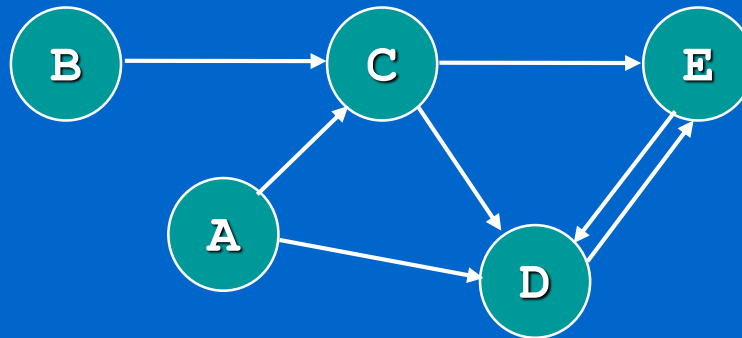
การแทนด้วยลิสต์ประชิด

- จุดยอดในกราฟส่วนใหญ่แล้วจะมีขอบเชื่อมไปยังจุดอื่นเป็นจำนวนไม่มาก ค่าของแต่ละองค์ประกอบโดยส่วนมากของเมตริกซ์ประชิดที่แทนกราฟจึงมีค่าเป็นเท็จ ทำให้เมตริกซ์มีลักษณะหริ่อมแหริ่ม (Sparse) ด้วยเหตุนี้เราจึงนิยมใช้โครงสร้างอีกรูปแบบหนึ่งที่เรียกว่า ลิสต์ประชิด (Adjacency List) มาแทนกราฟ ในลิสต์ประชิด จุดยอด x จุดใดๆในกราฟจะถูกแทนด้วยหัวลิสต์และ โหนดในลิงค์ลิสต์จะแทนขอบจาก x ไปยังจุดนั้น หัวลิสต์นี้มักนิยมเก็บรวมในรูปของอะเรย์

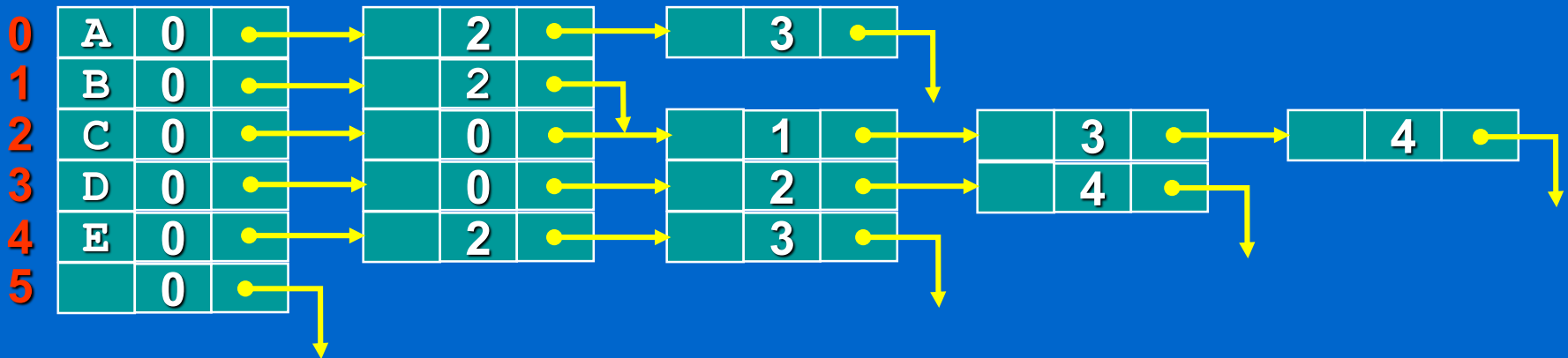
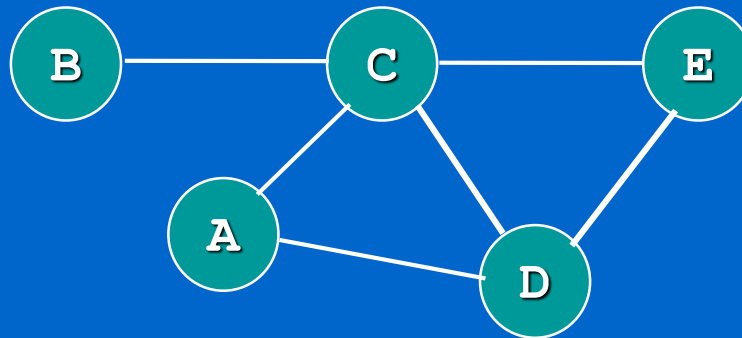
การแทนด้วยลิสต์ประชิด

```
typedef struct Node {  
    char          Info;  
    int           Link;  
    struct Node *Next;  
} TYPE_NODE;  
typedef struct Node *TYPE_NODEPTR;  
  
TYPE_NODE HeadNode[10];
```

การแทนด้วยลิสต์ประชิด



การแทนด้วยลิสต์ประชิด



การแทนด้วยลิสต์ประชิด

- ในโครงสร้างแบบลิสต์ประชิด เราจะสังเกตเห็นว่าลำดับการแทรกขอบเข้าลิสต์จะมีผลต่อลักษณะของลิงก์ลิสต์ ดังนั้นกราฟรูปเดียวกันอาจมีลิสต์ประชิดรูปร่างที่แตกต่างกันออกไป และจะมีผลกระทบต่อไปถึงการประมวลผล ในกรณีที่การประมวลผลไม่ขึ้นอยู่กับลำดับการวางตัวของขอบในลิสต์คำตอบที่ถูกต้องการประมวลผลจึงอาจจะมีได้หลายคำตอบ

-
-
-
-
-
-
-
-
-
-

การท่องเที่ยวในกราฟ

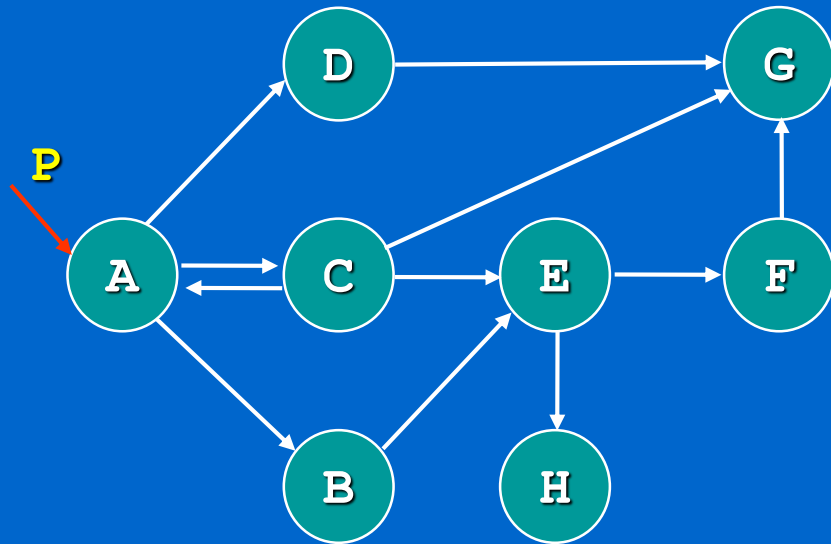
การท่องเที่ยวในกราฟ

- การประมวลผลกราฟจะอาศัยการท่องเที่ยวในกราฟอย่างมีแบบแผนเพื่อเยี่ยมชมจุดยอด ขั้นตอนวิธีมาตรฐานในการท่องเที่ยวในกราฟมี 2 วิธีคือ
 1. การค้นหาทางลึกก่อน (Depth-First Search)
 2. การค้นหาทางกว้างก่อน (Breadth-First Search)

-
-
-
-
-
-
-
-
-
-

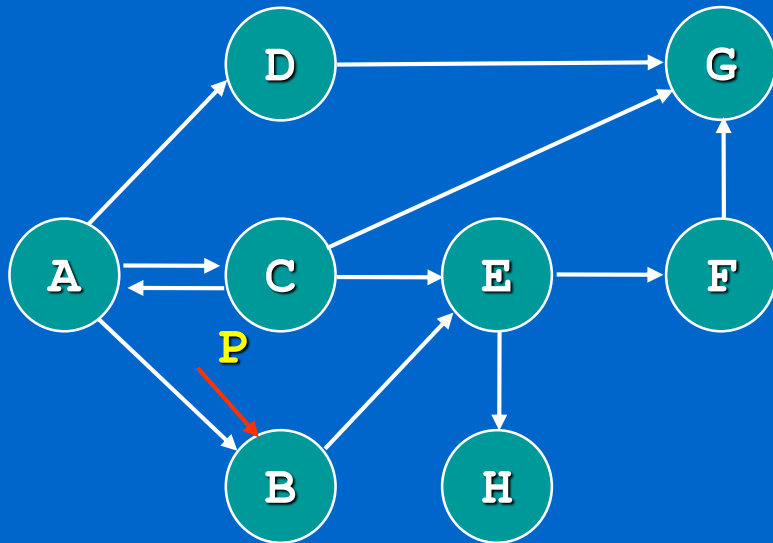
การค้นหาทางลึกก่อน

การค้นหาทางลึกก่อน



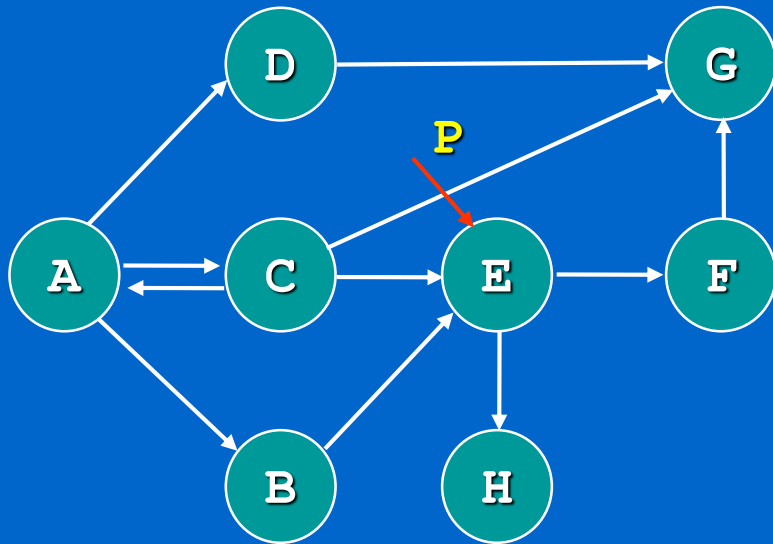
A

การค้นหาทางลึกก่อน



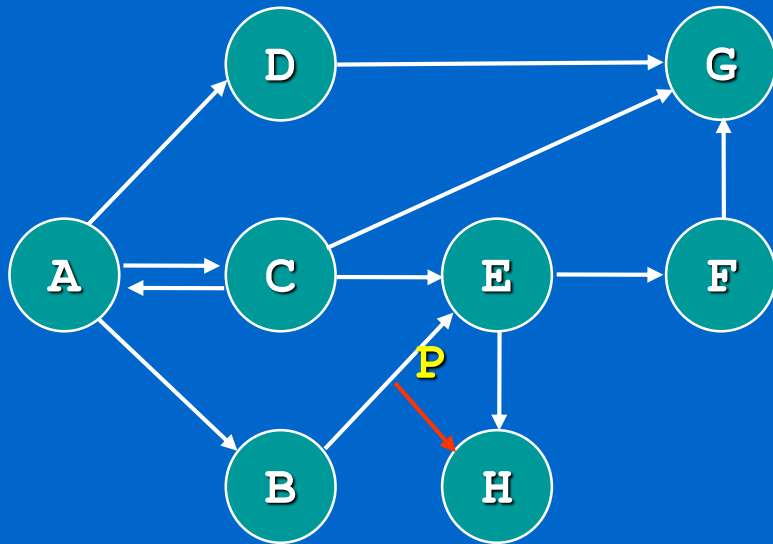
A, B

การค้นหาทางลึกก่อน



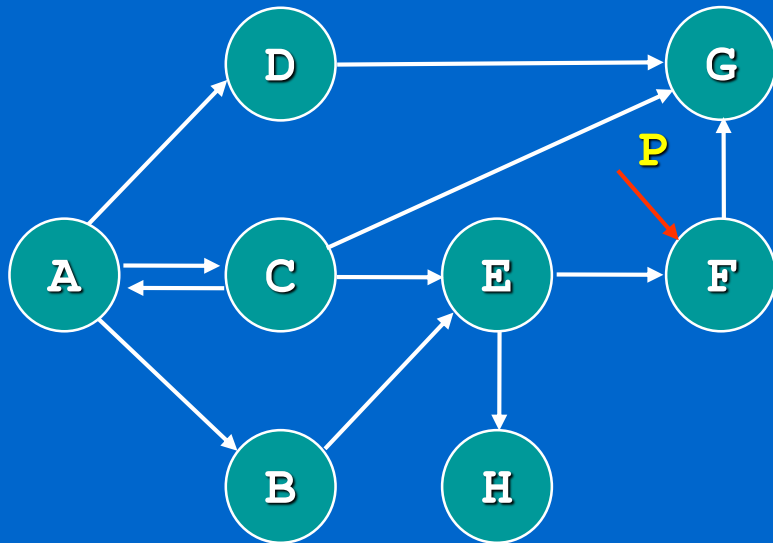
A, B, E

การค้นหาทางลึกก่อน



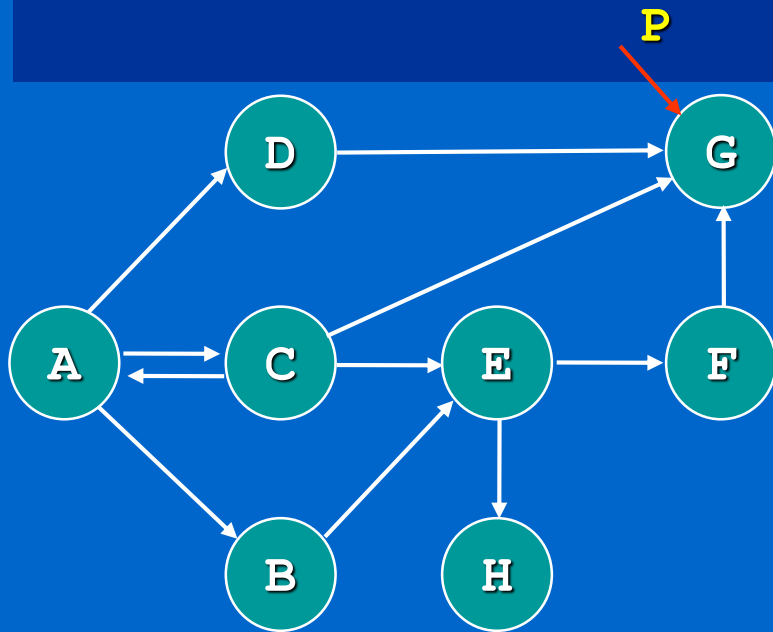
A, B, E, H

การค้นหาทางลึกก่อน



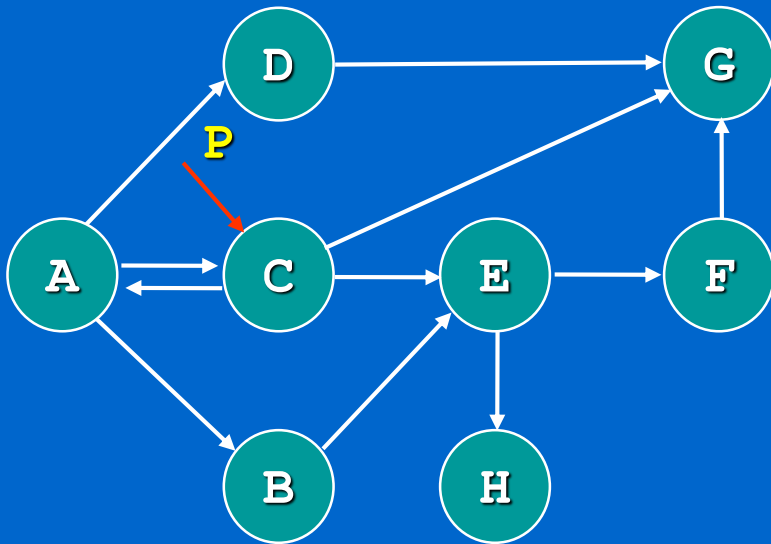
A, B, E, H, F

การค้นหาทางลึกก่อน



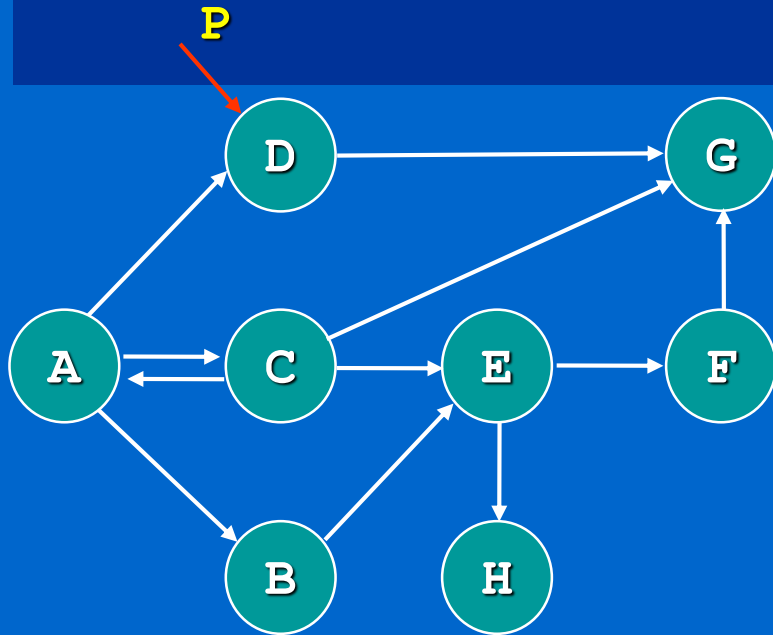
A, B, E, H, F, G

การค้นหาทางลึกก่อน



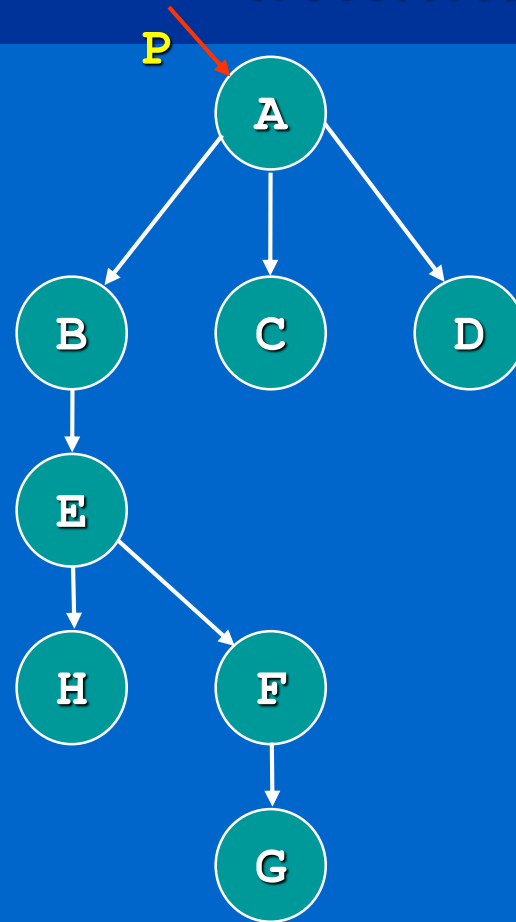
A, B, E, H, F, G, C

การค้นหาทางลึกก่อน



A, B, E, H, F, G, C, D

การค้นหาทางลึกก่อน



A, B, E, H, F, G, C, D

การค้นหาทางลึกก่อน

- ในกรณีที่เริ่มต้นท่องไปในกราฟที่จุดอื่น ลำดับการท่องอาจแตกต่างกันได้ ตัวอย่างเช่นหากเริ่มต้นที่จุด C จะได้ลำดับการเยี่ยมชม C, A, B, E, H, F, G, D หากเริ่มที่ B จะได้ลำดับการเยี่ยมชม B, E, H, F, G ซึ่งจะเห็นว่าการเยี่ยมชมเพียง 5 จุด จุดส่วนที่เหลือจะไม่มี การเยี่ยมชม เนื่องจากกราฟเป็นกราฟต่อเนื่องไม่แท้จริง การค้นหาทางลึกจะเยี่ยมชมจุดใดๆแล้วหาโหนดถัดไปเพื่อเยี่ยมชมด้วยวิธีค้นหาทางลึกอีก ขั้นตอนวิธีจึงสามารถเขียนอยู่ในรูปของชุดคำสั่งแบบรีเคอร์ซีฟได้ กระบวนคำสั่ง DFS() ต่อไปนี้เป็นชุดคำสั่งการค้นหาทางลึกสำหรับกราฟที่แทนด้วยลิสต์ประชิด

การค้นหาทางลึกก่อน

- เนื่องจากจุดในกราฟสามารถมีจุดประชิดกับจุดอื่นได้หลายจุดและอาจมีวัฏจักรซึ่งทั้งสองกรณีอาจทำให้การค้นหาทางลึกวนกลับมาพบจุดเดิม
อะเรย์ Status ในชุดคำสั่งจะหน้าที่เป็นแฟล็กประจำของจุดยอดแต่ละจุด
เพื่อคอยป้องกันไม่ให้เกิดการเยี่ยมชมซ้ำจุดที่เคยเยี่ยมชมไปแล้ว แฟล็กประจำจุด
ยอดทุกจุดจะมีค่าเริ่มต้นเป็นศูนย์ ภายหลังจากการเยี่ยมชมจุดใดๆด้วย
กระบวนการคำสั่ง Visit() แฟล็กประจำจุดจะถูกปรับให้มีค่าเป็น 1 ขั้นตอน
วิธีข้างต้นสามารถนำมาดัดแปลงใช้สำหรับกราฟที่แทนด้วยเมตริกซ์
ประชิดด้วยการใช้ชุดคำสั่ง DFS() ดังนี้

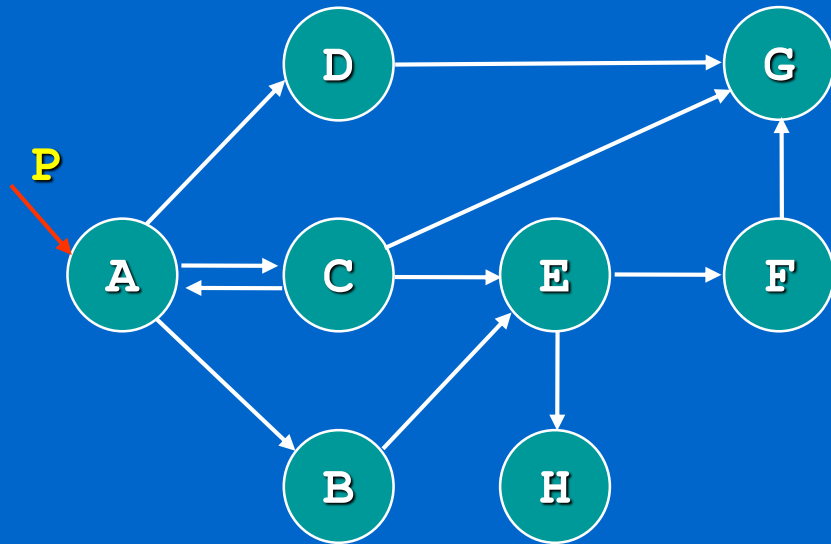
การค้นหาทางลัดก่อน

- ชุดคำสั่งการท่องไปในกราฟแบบค้นหาทางลัดก่อนในรูปรีเคอร์ซีฟสามารถดัดแปลงให้อยู่ในรูปธรรมดาโดยใช้สแตกเข้าช่วย เมื่อเยี่ยมชมจุดใดๆ แล้วให้เก็บรักษาจุดประชิดทุกจุดลงสู่สแตกแล้วดึงจุดประชิดออกมาทีละจุดเพื่อปฏิบัติการเยี่ยมชม หากจุดที่นำออกมาจากสแตก มีจุดประชิดอีกก็ให้ผลักจุดประชิดทุกจุดลงสู่สแตกอีก ทำเช่นนี้จนกว่าสแตกจะว่าง

-
-
-
-
-
-
-
-
-
-

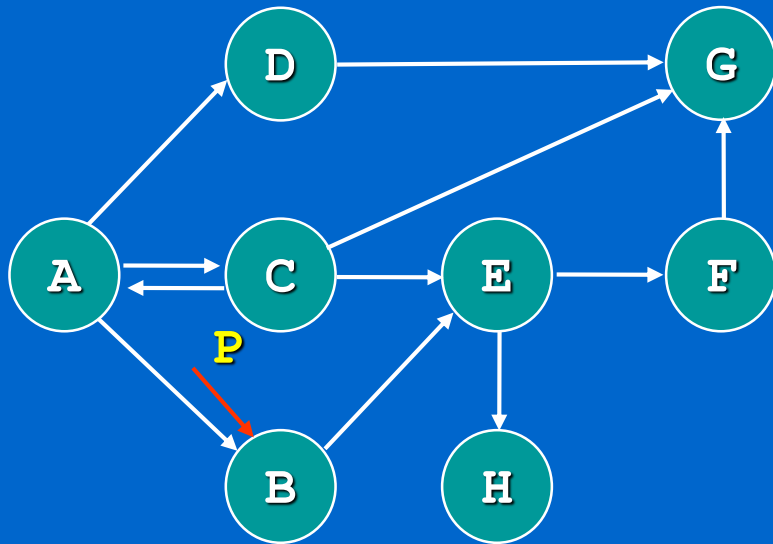
การค้นหาทางกว้างก่อน

การค้นหาทางกว้างก่อน



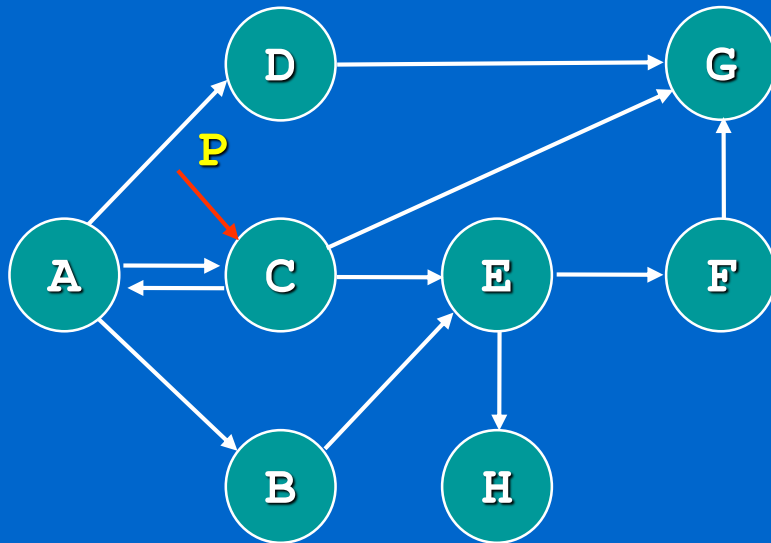
A

การค้นหาทางกว้างก่อน



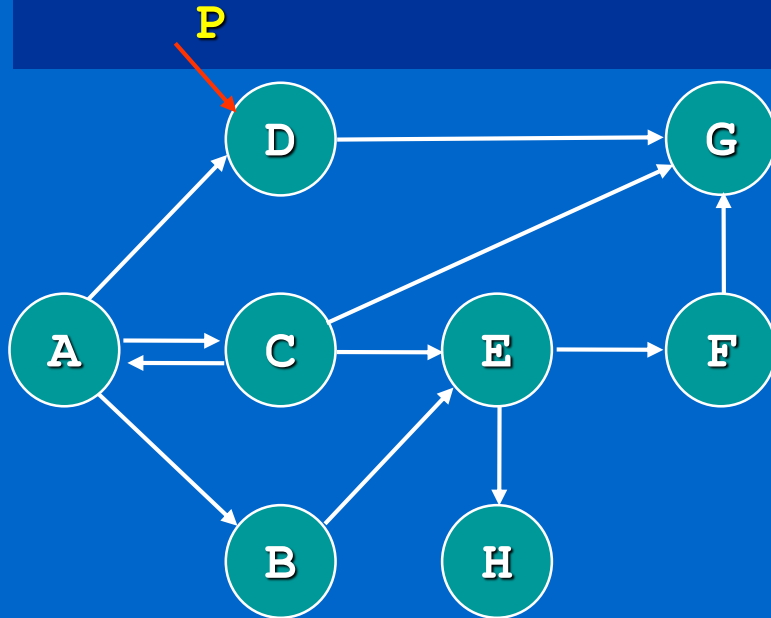
A, B

การค้นหาทางกว้างก่อน



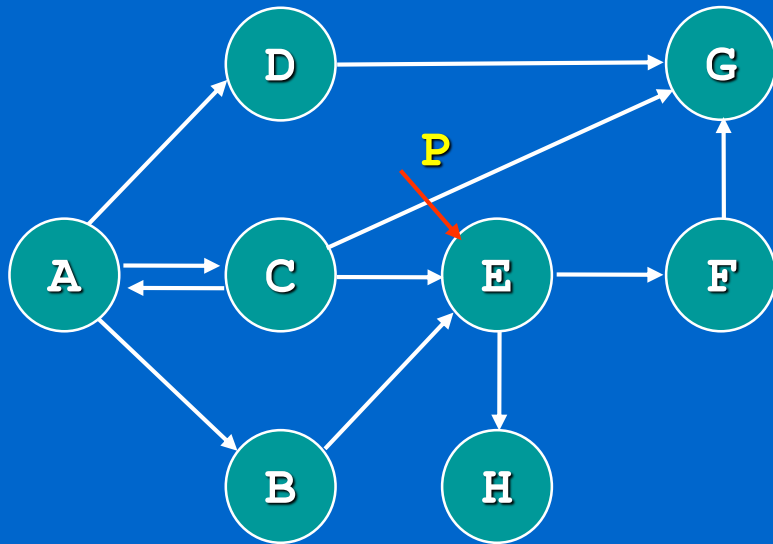
A, B, C

การค้นหาทางกว้างก่อน



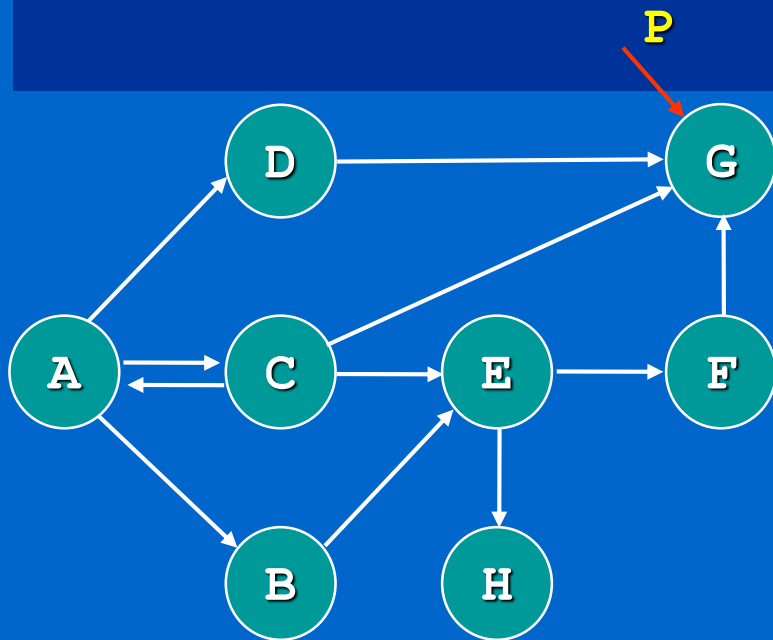
A, B, C, D

การค้นหาทางกว้างก่อน



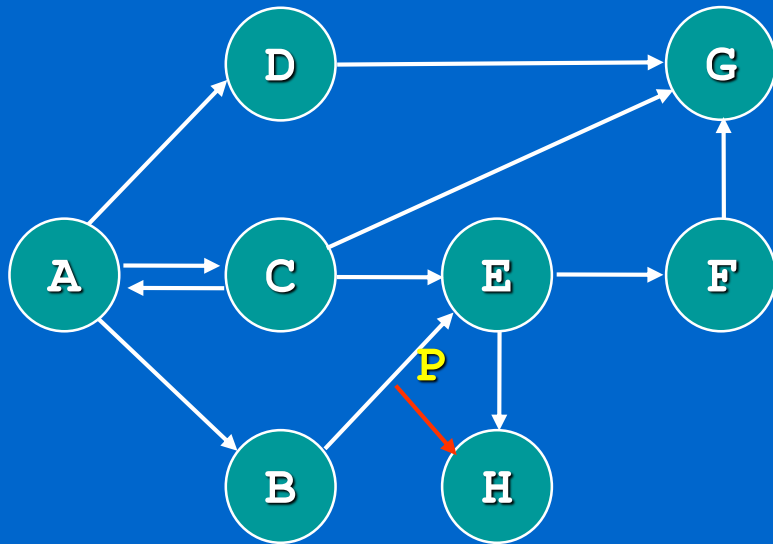
A, B, C, D, E

การค้นหาทางกว้างก่อน



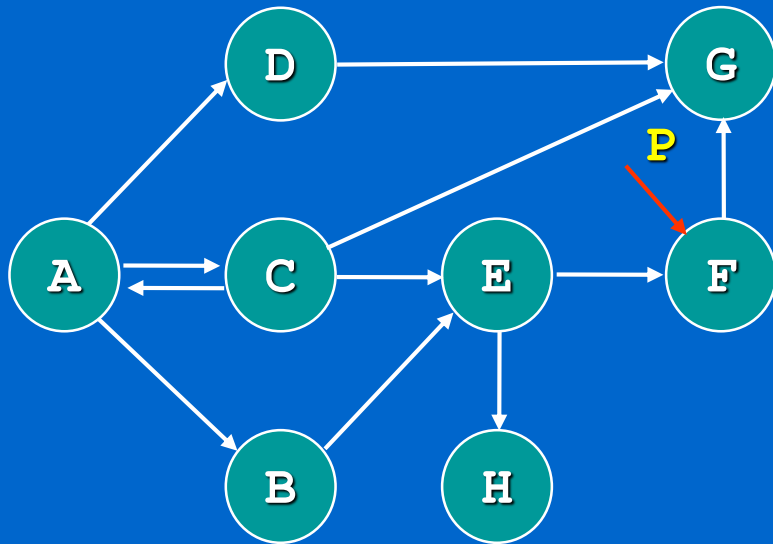
A, B, C, D, E, G

การค้นหาทางกว้างก่อน



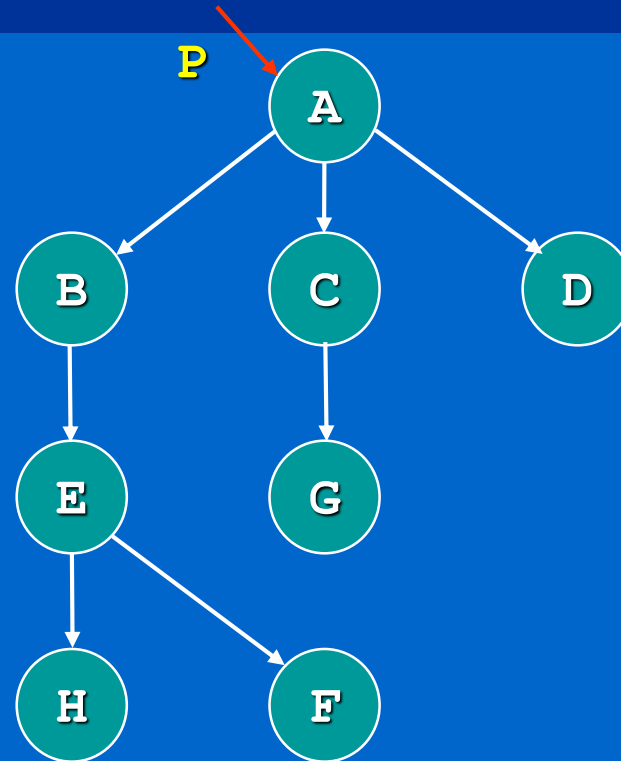
A, B, C, D, E, G, H

การค้นหาทางกว้างก่อน



A, B, C, D, E, G, H, F

การค้นหาทางกว้างก่อน



A, B, C, D, E, G, H, F

การท่องทางกว้างก่อน

- การค้นหาทางกว้างก่อนจะเยี่ยมชมตามลำดับก่อนหลัง กลไกที่จำเป็นต้องใช้ในการเก็บจุดยอดเพื่อรอการเยี่ยมชมจึงเข้าลักษณะเข้าก่อนออกก่อน ชุดคำสั่งการค้นหาทางกว้างจึงใช้คิวเพื่อเก็บจุดยอดที่รอคอยการเยี่ยมชม กระบวนคำสั่ง BFS() ต่อไปนี้เป็นชุดคำสั่งการค้นหาทางกว้างสำหรับกราฟที่แทนด้วยลิสต์ประชิด

การท่องเที่ยวกว้างก่อน

- ในทำนองเดียวกันกับการค้นหาในทางลึกอะเรย์ Status ในชุดคำสั่งจะหน้าที่เป็นแฟล็กประจำของจุดยอดแต่ละจุดเพื่อคอยป้องกันไม่ให้เกิดการเยี่ยมชมซ้ำจุดที่เคยเยี่ยมชมไปแล้ว แฟล็กประจำจุดยอดทุกจุดจะมีค่าเริ่มต้นเป็นศูนย์ ภายหลังจากการเยี่ยมชมจุดใดๆด้วยกระบวนการคำสั่ง Visit() แฟล็กประจำจุดจะถูกปรับให้มีค่าเป็น 1

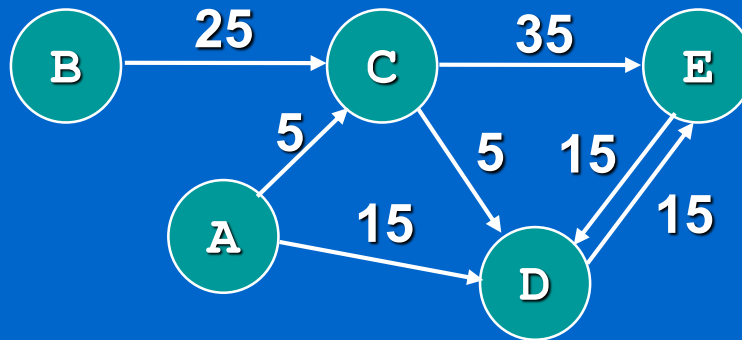
-
-
-
-
-
-
-
-
-
-

กราฟถ่วงน้ำหนัก (Weighted Graph)

กราฟถ่วงน้ำหนัก (Weighted Graph)

- กราฟที่กล่าวไปแล้วในหัวข้ออื่นๆ เป็นกราฟที่มีข่าวสารประจำอยู่เฉพาะที่จุดยอด แต่ยังมีกราฟอีกประเภทหนึ่งที่มีข่าวสารประจำอยู่ที่ขอบของกราฟอีกด้วย ข่าวสารประจำอยู่ที่ขอบของกราฟนี้จะเป็นตัวเลขแสดงความสัมพันธ์ระหว่างจุดประจำขอบนั้น ค่าตัวเลขนี้เรียกว่า **น้ำหนัก (Weight)** กราฟที่มีค่าน้ำหนักกำกับก็จะเรียกว่า **กราฟถ่วงน้ำหนัก (Weighted Graph)** เมตริกซ์ประชิดสามารถนำมาใช้แทนกราฟถ่วงน้ำหนักได้โดยใช้ค่าน้ำหนักแสดงถึงขอบที่เชื่อมจุด ส่วนขอบที่ไม่ปรากฏระหว่างจุดก็ให้แทนด้วยตัวเลข 0 หรือค่าติดลบ

กราฟถ่วงน้ำหนัก (Weighted Graph)



A	0
B	1
C	2
D	3
E	4

	0	1	2	3	4
0	0	0	5	15	0
1	0	0	25	0	0
2	0	0	0	5	35
3	0	0	0	0	15
4	0	0	0	15	0

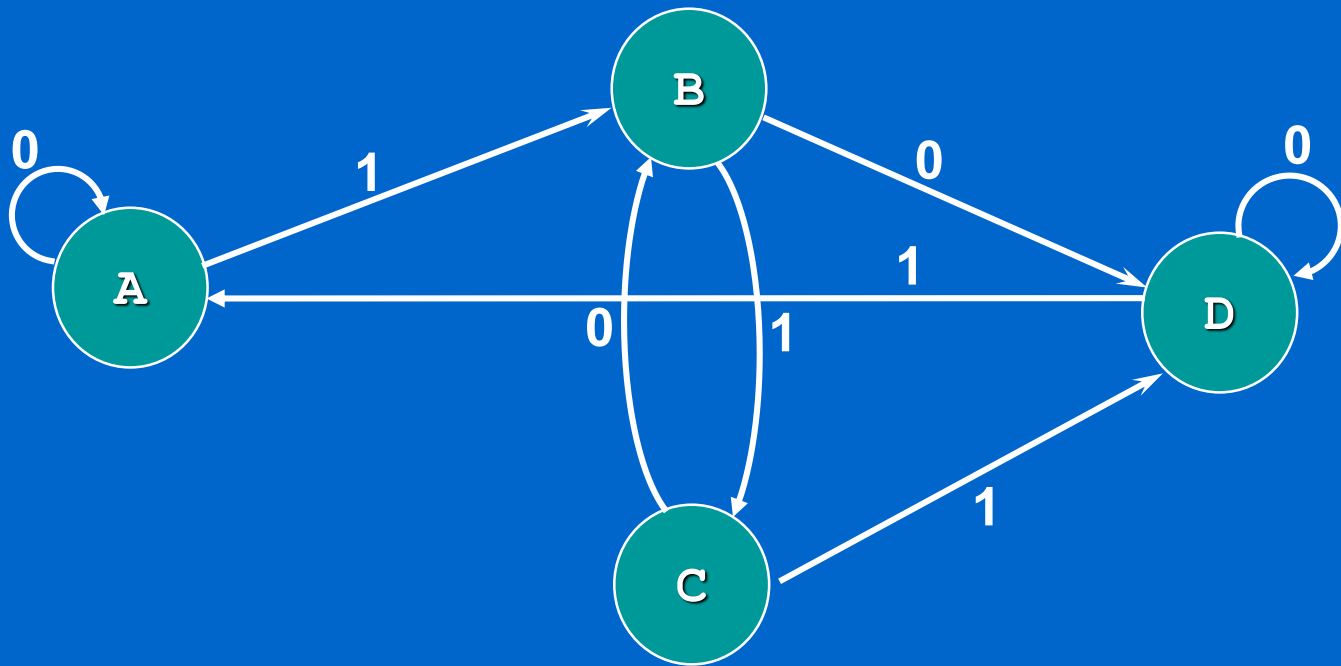
กราฟสถานะ (State Graph)

- คำน่านักประจำขอบ อาจะหมายถึงระยะห่างระหว่างเมืองเมื่อเราใช้กราฟแทนแผนที่ หรือหมายถึงระยะเวลาที่ใช้ในเพื่อปฏิบัติงานหนึ่งไปสู่งานเป้าหมายอีกงานหนึ่งเมื่อเราใช้กราฟแทนโครงการหรือแผนงานตามปกติแล้วค่าน่านักประจำขอบในกราฟถ่วงน้ำหนักจะเป็นไปได้โดยไม่จำกัด แต่จะมีกราฟถ่วงน้ำหนักพิเศษประเภทหนึ่งจะมีค่าน่านักที่จำกัดด้วยเซตของตัวเลขหรือตัวอักษรเซตหนึ่งๆกราฟถ่วงน้ำหนักประเภทนี้เรียกว่า**กราฟสถานะ** (State Graph)

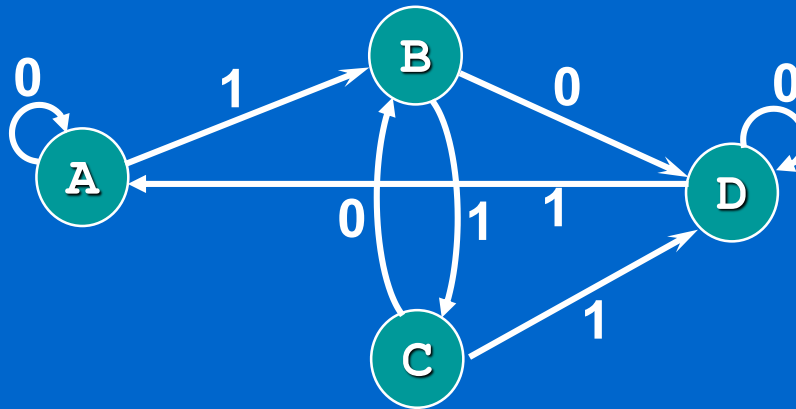
กราฟสถานะ (State Graph)

- จุดยอดแต่ละจุดในกราฟสถานะจะแทนสถานะหนึ่งและค่าน้ำหนักจะเป็นตัวกำหนดการเปลี่ยนแปลงจากสถานะหนึ่งไปสู่อีกสถานะหนึ่ง ดังเช่นกราฟในรูป สมมติว่าสถานะปัจจุบันอยู่ที่ A หากมีข้อมูลเข้ามาเป็น 0 จะยังคงในสถานะ A โดยไม่มีการเปลี่ยนแปลง (สังเกตได้จากวงรอบตัวเองที่ A) แต่ถ้ามีข้อมูลเข้ามาเป็น 1 จะมีการเปลี่ยนไปเป็นสถานะ B การแทนกราฟสถานะอาจใช้อะเรย์ 2 มิติ โดยในแนวแถวแทนจุดในกราฟและแนวคอลัมน์แทนค่าน้ำหนัก ข้อมูลในอะเรย์ก็คือจุดที่สมนัยกับค่าข้อมูลเข้า และมีชื่อเรียกว่า**ตารางสถานะ** (State Table)

กราฟสถานะ (State Graph)

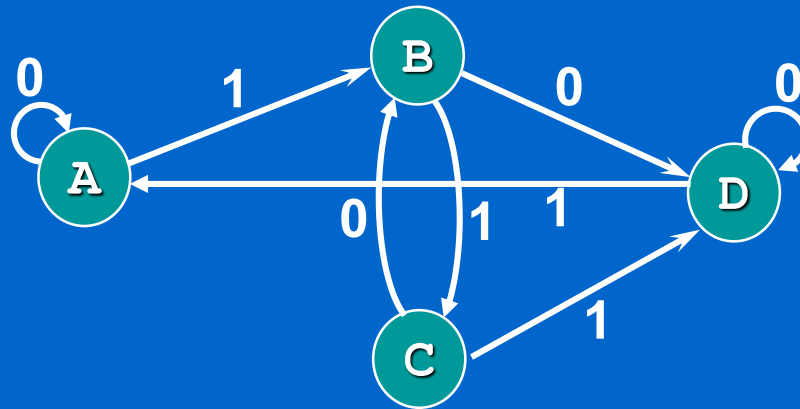


กราฟสถานะ (State Graph)



	0	1
A	A	B
B	D	C
C	B	D
D	D	A

กราฟสถานะ (State Graph)



A	0
B	1
C	2
D	3

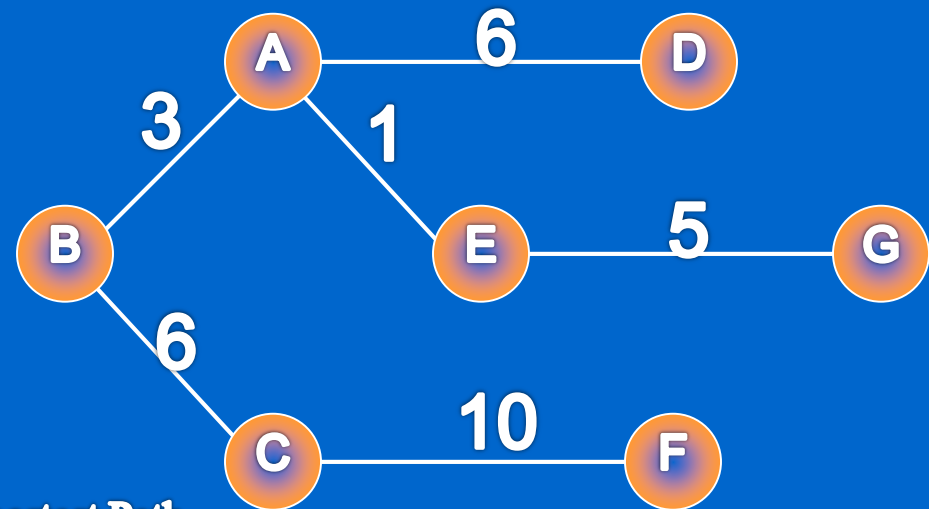
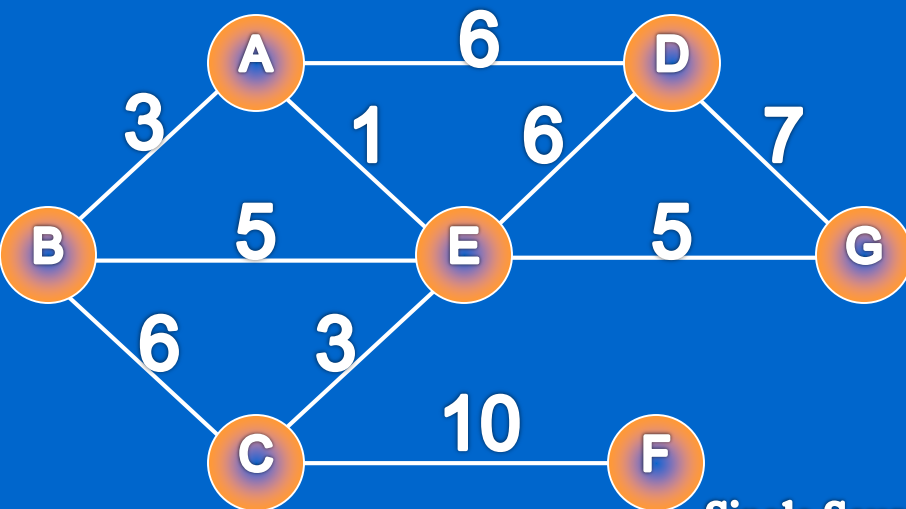
	0	1
0	0	1
1	3	2
2	1	3
3	3	0

กราฟสถานะ (State Graph)

- กราฟสถานะนำไปประยุกต์ใช้ในงาน**ทฤษฎีการคำนวณ** (Theory of Computation) การออกแบบระบบฮาร์ดแวร์ หรือในระบบแปลภาษา (Translator หรือ Compiler) เป็นต้น
- การท่องเที่ยวในกราฟแบบค้นหาทางลึกก่อนเป็นรูปแบบทั่วไปของการท่องเที่ยวในต้นไม้แบบตามลำดับเมื่อเปรียบเทียบการท่องเที่ยวในทั้งสองโครงสร้างแล้วจะพบว่ามีส่วนที่เหมือนกันคือเมื่อเดินผ่านจุดใดก็จะเยี่ยมชมจุดนั้นแล้วจึงค้นหาจุดที่อยู่ถัดไป

Single Source Shortest Path

- หรือ Dijkstra's Algorithm / Dijkstra เป็นนักคณิตศาสตร์ชาวดัตช์
- ใช้หาเส้นทางจาก **โหนดเริ่มต้น** ไปยังโหนดใดๆ ให้มีระยะทางที่สั้นที่สุด
- มีประโยชน์ ในงานพวกขนส่งสินค้า เช่น ต้องการขนส่งจาก **จุดหนึ่ง** ไปยังจุดหนึ่งด้วยทางที่สั้นที่สุด



Single Source Shortest Path
(พิจารณาที่ B)

Single Source Shortest Path

- Algorithm นี้คล้ายกับ Breadth-first Search
- ปรับปรุงนิดหน่อยคือ
 - ใช้คิวชนิด Priority Queue
 - ใช้เส้นทางรวมจากจุดเริ่มต้นมายังโหนดนั้นๆ เพื่อระบุ Priority ของสมาชิกที่อยู่ในคิว
- ใช้ได้กับค่า Edge ที่มีค่าตั้งแต่ 0 ขึ้นไป

Single Source Shortest Path

1. PriorityQueue = {startVertex} /*โครงสร้าง queue แบบมีอภิสิทธิ์*/

2. Until PriorityQueue ว่าง Do

1.1 ดึงสมาชิกตัวแรกใน Queue ออกมา(ให้ชื่อว่า X)

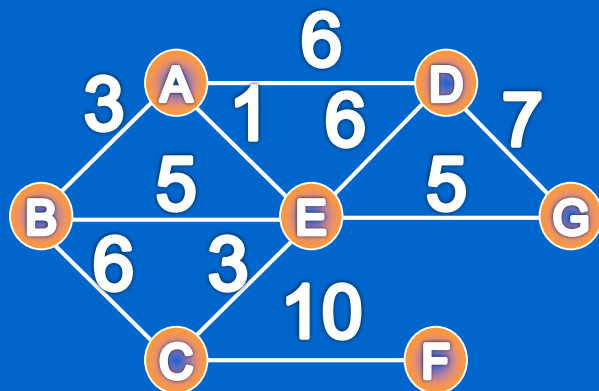
1.2 หาก X เคยถูกเลือกเป็นเส้นทางแล้ว กลับไปข้อ 1.1

1.3 เลือกโหนด X ให้เป็นเส้นทางจริง

1.4 สำหรับโหนดใดๆ ที่เชื่อมต่อกับ X ให้ทำดังนี้

1.4.1 คำนวณระยะทางรวมของโหนด X มายังโหนดนั้นๆ

1.4.2 นำทุกเส้นทางไปไว้ใน PriorityQueue



รอบที่ 1

Step

Queue

Graph

1. PriorityQueue = {startVertex}

PQ = {(B,B,0)}

2. ดึงสมาชิกตัวแรกออกมา

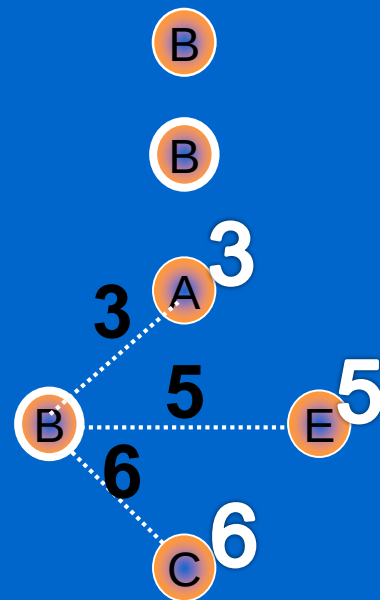
PQ = {}

3. เลือก B ให้เป็นเส้นทางจริง

PQ = {}

4. สำหรับทุกโหนดที่ต่อกับ B คำนวณระยะทางแล้วนำกลับไปเก็บใน PQ

PQ = {(B,A,3), (B,E,5), (B,C,6)}



Step

Queue

Graph

5. ดึง (B,A,3) สมาชิกตัวแรกออกมา

$PQ = \{(B,E,5), (B,C,6)\}$

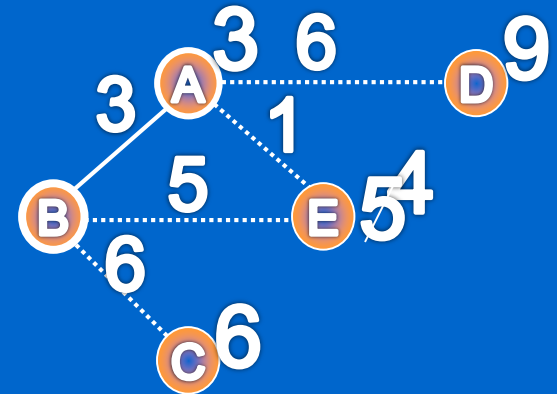


6. เลือก A ให้เป็นเส้นทางจริง

$PQ = \{(B,E,5), (B,C,6)\}$

7. สำหรับทุกโหนดที่ต่อกับ A คำนวณระยะทางแล้วนำกลับไปเก็บใน PQ

$PQ = \{(A,E,4), (B,E,5), (B,C,6), (A,D,9)\}$



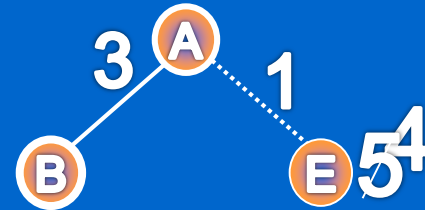
Step

8. ดึง(A,E,4)สมาชิกตัวแรกออกมา

Queue

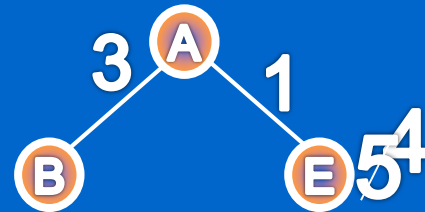
PQ={ (B,E,5), (B,C,6), (A,D,9) }

Graph



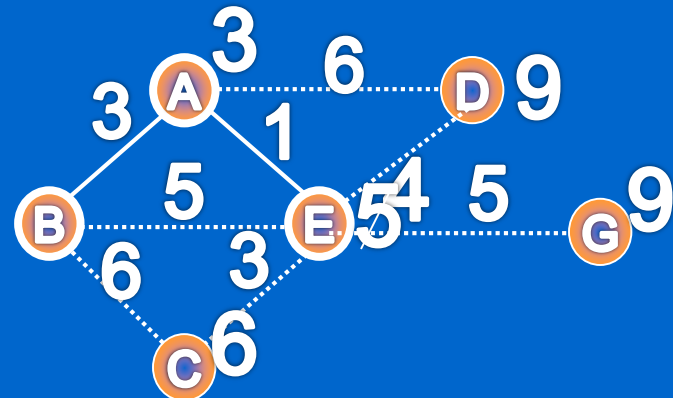
9. เลือก E ให้เป็นเส้นทางจริง

PQ={ (B,E,5), (B,C,6), (A,D,9) }



10. สำหรับทุกโหนดที่ต่อกับ E คำนวณระยะทางแล้วนำกลับไปเก็บใน PQ

PQ={ (B,E,5), (B,C,6), (E,C,7), (A,D,9), (E,G,9), (E,D,10) }

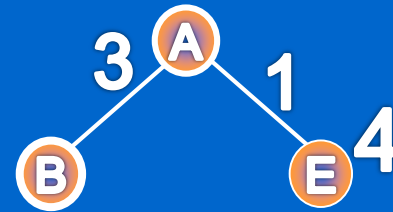


StepQueueGraph

11. ดึง(B,E,5) สมาชิกตัวแรกออกมา

$PQ = \{(B,C,6), (E,C,7), (A,D,9), (E,G,9), (E,D,10)\}$

แต่เนื่องจาก (B,E,5) โหนด E ถูกเลือกเป็น
เส้นทางหลักไปแล้ว ดังนั้นจึงไม่สนใจ



Step

12. ดึง(B,C,6) สมาชิกตัวแรกออกมา

13. เลือก C ให้เป็นเส้นทางจริง

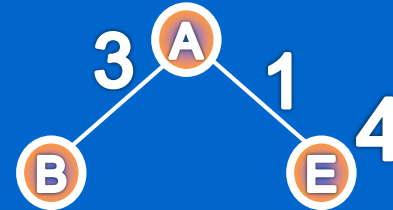
14. สำหรับทุกโหนดที่ต่อกับ C คำนวณ
ระยะทางแล้วนำกลับไปเก็บใน PQ

$PQ = \{(E,C,7)(A,D,9),(E,G,9)(E,D,10),(C,F,16)\}$

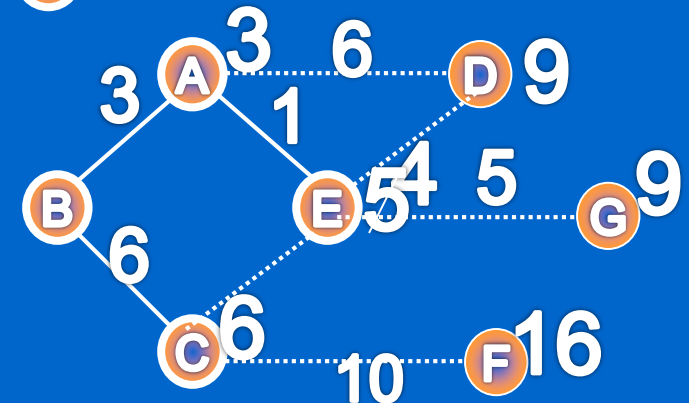
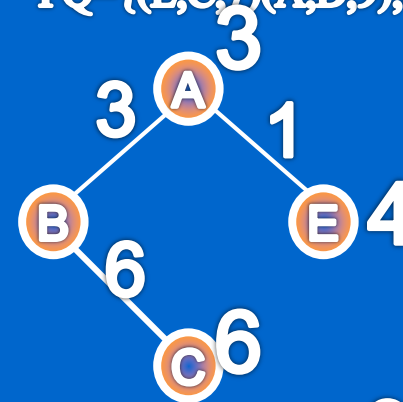
หมายเหตุ (C,E,9)ไม่ต้อง Add เพราะ E ถูกเลือกไปแล้ว

Queue

$PQ = \{(E,C,7)(A,D,9),(E,G,9)(E,D,10)\}$



$PQ = \{(E,C,7)(A,D,9),(E,G,9)(E,D,10)\}$



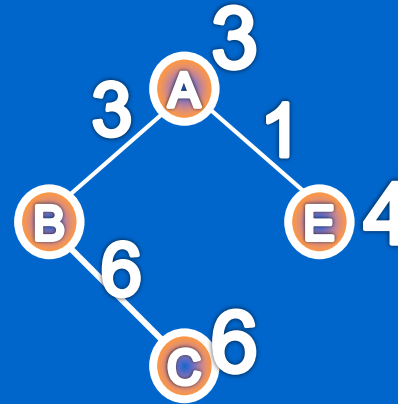
Step

Queue

15. ดึง(E,C,7) สมาชิกตัวแรกออกมา

$PQ = \{(A,D,9), (E,G,9), (E,D,10), (C,F,16)\}$

แต่เนื่องจาก (E,C,7) โหนด C ถูกเลือกเป็น
เส้นทางหลักไปแล้ว ดังนั้นจึงไม่สนใจ



Step

Queue

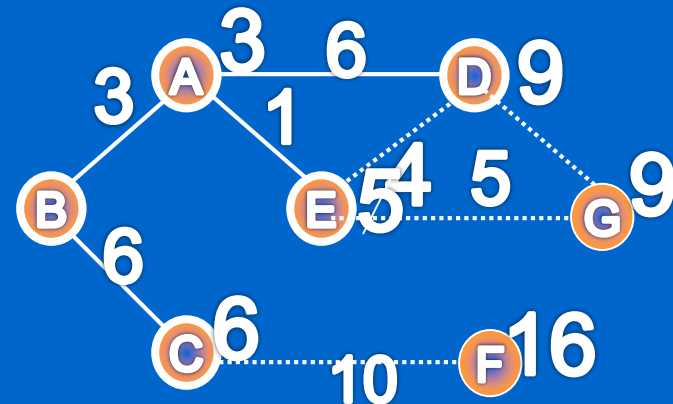
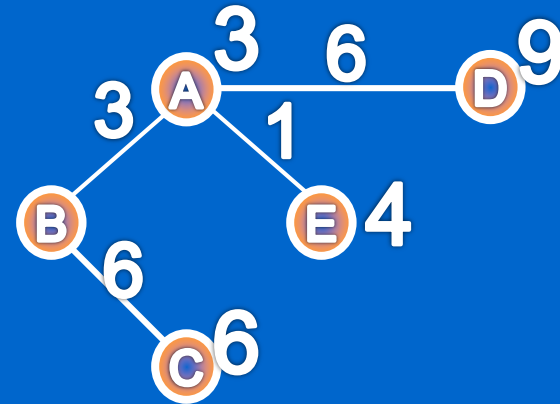
16. ดึง (A,D,9) สมาชิกตัวแรกออกมา

$PQ = \{(E,G,9), (E,D,10), (C,F,16)\}$

17. เลือก D ให้เป็นเส้นทางจริง

18. สำหรับทุกโหนดที่ต่อกับ D คำนวณ
ระยะทางแล้วนำกลับไปเก็บใน PQ

$PQ = \{(E,G,9), (E,D,10), (C,F,16), (D,G,16)\}$



Step

Queue

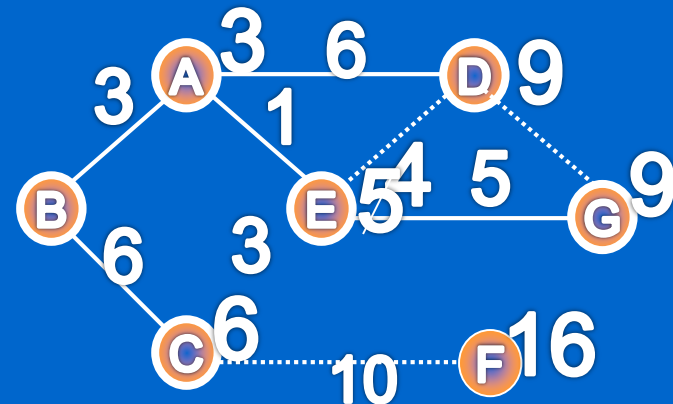
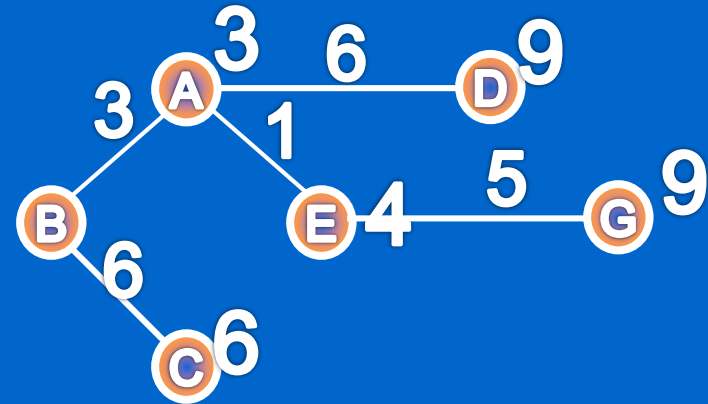
19. ดึง (E,G,9) สมาชิกตัวแรกออกมา

$PQ = \{(E,D,10), (C,F,16), (D,G,16)\}$

20. เลือก G ให้เป็นเส้นทางจริง

21. สำหรับทุกโหนดที่ต่อกับ G คำนวณ
ระยะทางแล้วนำกลับไปเก็บใน PQ

$PQ = \{(E,D,10), (C,F,16), (D,G,16)\}$



Step

Queue

Graph

22. ดึง (E,D,10) สมาชิกตัวแรกออกมา

$PQ = \{(C,F,16), (D,G,16)\}$

แต่เนื่องจาก (E,D,7) โหนด D ถูกเลือกเป็นเส้นทางหลักไปแล้ว ดังนั้นจึงไม่สนใจ

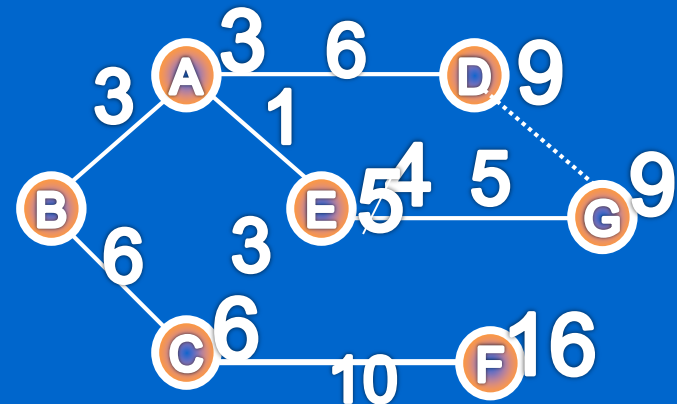
23. ดึง (C,F,16) สมาชิกตัวแรกออกมา

$PQ = \{(D,G,16)\}$

24. เลือก F ให้เป็นเส้นทางจริง

25. สำหรับทุกโหนดที่ต่อกับ F คำนวณระยะทางแล้วนำกลับไปเก็บใน PQ (ไม่มีแล้ว)

รอบที่ 10



Step

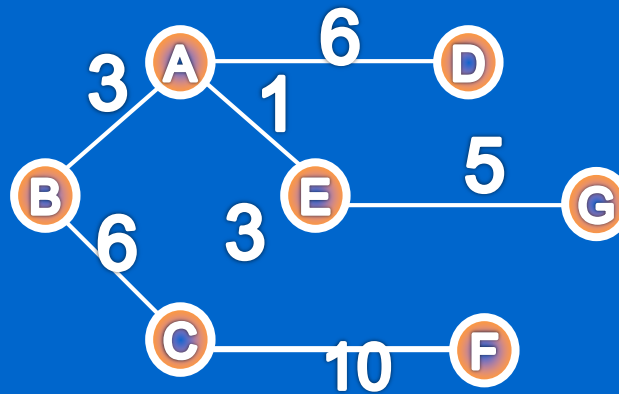
Queue

26. ดึง (D,G,16) สมาชิกตัวแรกออกมา

PQ={}

แต่เนื่องจาก (D,G,16) โหนด G ถูกเลือกเป็น
เส้นทางหลักไปแล้ว ดังนั้นจึงไม่สนใจ

27. เมื่อ PriorityQueue ว่าง จึงจบการทำงาน



Question & Answer

